P3179R1
C++ parallel range algorithms

Published Proposal, 2024-05-22

This version:
https://wg21.link/P3179R1

Authors:
Ruslan Arutyunyan (Intel)
Alexey Kukanov (Intel)

Audience:
SG9, SG1

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes adding parallel algorithms that work together with the C++ Ranges library.

Table of Contents

§ 1. Motivation

Standard parallel algorithms with execution policies which set semantic requirements to user-provided callable objects were a good start for supporting parallelism in the C++ standard.

The C++ Ranges library - ranges, views, etc. - is a powerful facility to produce lazily evaluated pipelines that can be processed by range-based algorithms. Together they provide a productive and expressive API with the room for extra optimizations.

Combining these two powerful features by adding support for execution policies to the range-based algorithms opens an opportunity to fuse several computations into one parallel algorithm call, thus reducing the overhead on parallelism. That is especially valuable for heterogeneous implementations of

parallel algorithms, for which the range-based API helps reducing the number of kernels submitted to an accelerator.

Earlier, [P2500R2] proposed to add the range-based C++ parallel algorithms together with its primary goal of extending algorithms with schedulers. We have decided to split those parts to separate papers, which could progress independently.

This paper is targeted to C++26.

§ 2. Design overview

This proposal addresses absence of execution policy support for C++ range-based algorithms. In the nutshell, the proposal extends C++ range algorithms with overloads taking any standard C++ execution policy as a function parameter. These overloads are further referred to as *parallel range algorithms*.

§ 2.1. Design summary

- The parallel range algorithms should be close to C++17 classic ones to use the code with minimal required changes.
- The parallel range algorithms should return the same type as the corresponding serial range algorithms, unless there are strong reasons for a different choice.
- The proposed algorithms should follow the design of serial range algorithms with regard to name lookup.
- The parallel range algorithms should take `range` as an output for the overloads with ranges, and additionally take an output sentinel for the "iterator + sentinel" overloads.
- The required range and iterator categories should be random access, until a better parallelism-friendly abstraction is proposed.
- The parallel range algorithms should require bounded ranges for both input and output.
- The proposed API should require callable object passed to an algorithm to be `regular_invocable` where possible.
- The proposed API is not a customization point.
- The proposed algorithms should follow the design of serial range algorithms with regard to `constexpr` support.

§ 2.2. Coexistence with schedulers

We believe that adding parallel range algorithms does not have the risk of conflict with anticipated scheduler-based algorithms, because an execution policy does not satisfy the requirements for a policy-aware scheduler ([P2500R2]), a sender ([P3300R0]), or really anything else from [P2300R9] that can be used to specify such algorithms.

At this point we do not, however, discuss how the appearance of schedulers may or should impact the execution rules for parallel algorithms specified in [algorithms.parallel.exec], and just assume that the same rules apply to the range algorithms with execution policies.

§ 2.3. Switch to parallel range algorithms with minimal changes

One of the goals is to require a minimal amount of changes when switching from the existing API to parallel range algorithms.

The C++17 parallel `for_each` call:

```
std::for_each(std::execution::par, v.begin(), v.end(), [](auto& x) { ++x; }
```

can be changed to one of the following:

```
// Using an iterator and a sentinel
std::ranges::for_each(std::execution::par, v.begin(), v.end(), [](auto& x)

// Switching to use a range
std::ranges::for_each(std::execution::par, v, [](auto& x) { ++x; });
```

If serial range algorithms are used in the code, switching to parallel version would look like in the example below.

The C++20 range-based `for_each` call:

```
std::ranges::for_each(v, [](auto& x) { ++x; });
```

becomes:

```
std::ranges::for_each(std::execution::par, v, [](auto& x) { ++x; });
```

As you can see the changes are pretty simple:

- In the first case only the namespace is changed, and users might also change `v.begin()`, `v.end()` to just `v`.
- In the second case an execution policy is added as the first function argument. The same is true for the *Iterator* + *Sentinel* overload

§ 2.4. Algorithm return types

We explored possible algorithm return types and came to conclusion that returning the same type as serial range algorithms is the preferred option to make the changes for enabling parallelism minimal.

```
auto res = std::ranges::sort(v);
```

becomes:

```
auto res = std::ranges::sort(std::execution::par, v);
```

The only exception we are going to make is `std::ranges::for_each` and `std::ranges::for_each_n` because they have to take previous design decisions for algorithms into account.

Let's consider the following table:

API	Return type
`std::for_each`	`Fun`
Parallel `std::for_each`	`void`
`std::for_each_n`	`It`
Parallel `std::for_each_n`	`It`
`std::ranges::for_each`	`for_each_result<ranges::borrowed_iterator_t<R>, Fun>`
`std::ranges::for_each`, `I` + `S` overload	`for_each_result<I, Fun>`

`std::ranges::for_each_n`	`for_each_n_result<I, Fun>`

The return type for parallel `std::for_each` is `void` because it does not make sense (or might be even dangerous) to return a function object. The idea is that the function object is copyable (not just movable, like for serial `for_each`) for the parallelism sake. That implies that users cannot rely on any state accumulation within that function object because algorithm might have as many copies as it needs.

Based on the explanation and the feedback from SG9 we believe the most reasonable return type for `std::ranges::for_each` and for `std::ranges::for_each_n` can be summarized as following:

API	Return type
`std::for_each`	`Fun`
Parallel `std::ranges::for_each`	`ranges::borrowed_iterator_t<R>`
Parallel `std::ranges::for_each`, `I` + `S` overload	`I`
Parallel `std::ranges::for_each_n`	`I`

§ 2.5. Non ADL-discoverable functions

We believe the proposed functionality should have the same behavior as serial range algorithms regarding the name lookup. For now, the new overloads are supposed to be special functions that are not discoverable by ADL (the status quo of the standard for serial range algorithms).

[P3136R0] suggests to respecify range algorithms to be actual function objects. If adopted, that proposal will apply to all algorithms in the `std::ranges` namespace, thus automatically covering the parallel algorithms we propose.

Either way, adding parallel versions of the range algorithms should not be a problem. Please see § 4.1 Possible implementation of a parallel range algorithm for more information.

§ 2.6. Requiring `random_access_iterator` or `random_access_range`

C++17 parallel algorithms require *LegacyForwardIterator* for input data sequences. Although it might be useful for `std::execution::seq` policy, it does not make a lot of sense for an actual parallel implementation. We are not aware of an existing implementation supporting forward iterators well for any of `unseq`, `par` or `par_unseq` policies. oneAPI Data Parallel C++ library (oneDPL) supports forward iterators only for a very few algorithms, only for `par` and only in the implementation based on oneTBB.

Though the feedback we received in Tokyo requested to support forward ranges, we would like this question to be discussed in more detail. We believe that forward ranges and iterators are bad abstractions for parallel data processing, and allowing those would result in wrong expectations and unsatisfactory user experience with parallel algorithms.

There are two main reasons why others do not want to restrict parallel algorithms by only random access ranges:

- That would prevent some useful views, such as `filter_view`, from being used with parallel range algorithms.
- That would be inconsistent with the C++17 parallel algorithms.

Given the other aspects of the proposed design, we believe inconsistency with C++17 parallel algorithms is inevitable and should not become a gating factor for important design decisions.

The question of supporting the standard views that do not provide random access is very important. We think though that it should better be addressed through proper abstractions and new concepts added specifically for that purpose. We intend to work on developing these in a future revision of this paper or in another paper. For now though random access ranges with known boundaries (see § 2.7 Requiring ranges to be bounded) is the closest match we were able to find in the standard. Starting from that and gradually enabling other types of ranges in a source-compatible manner seems to us better than blanket allowance of any `forward_range`.

§ 2.7. Requiring ranges to be bounded

One of the requirements we want to put on the parallel range algorithms is to disallow use of unbounded sequences. The reasons for that are:

- First, for efficient parallel implementation we need to know the iteration space bounds. Otherwise, it's hard to apply the "divide and conquer" strategy for creating work for multiple execution threads.
- Second, while serial range algorithms allow passing an "infinite" range like `std::ranges::views::iota(0)`, it may result in an endless loop. It's hard to imagine usefulness of that in the case of parallel execution. Requiring data sequences to be bound potentially prevents errors at run-time.

We have evaluated a few options to specify such a requirement, and for now decided to use the `sized_sentinel_for` concept. It is sufficient for the purpose and at the same does not require any-

thing that a random access range would not already provide. For comparison, the `sized_range` concept adds a requirement of `std::ranges::size(r)` to be well-formed for a range `r`.

§ 2.8. Requirements for callable parameters

In [P3179R0] we proposed that parallel range algorithms should require function objects for predicates, comparators, etc. to have `const`-qualified `operator()`, with the intent to provide compile-time diagnostics for mutable function objects which might be unsafe for parallel execution. We have got contradictory feedback from SG1 and SG9 on that topic: SG1 preferred to keep the behavior consistent with C++17 parallel algorithms, while SG9 supported our design intent.

We did extra investigation and decided that requiring `const`-qualified operator at compile-time is not strictly necessary because:

- The vast majority of the serial range algorithms requires function objects to be `regular_invocable` (or its derivatives), which already has the semantical requirement of not modifying either the function object or its arguments. While not enforced at compile-time, it seems good enough for our purpose because it demands having the same function object state between invocations (independently of `const` qualifier), and it is consistent with serial range algorithms.
- Remaining algorithms, like `std::ranges::for_each` or `std::ranges::generate`, should be considered individually. For example, `for_each` having a mutable `operator()` is no more a big concern, because based on the SG9 poll in Tokyo we have dropped the function object from the `for_each` return type.

The following example works fine for serial code. While it compiles for parallel code, users should not assume that the semantics remains intact. Since the parallel version of `for_each` requires function object to be copyable, it is not guaranteed that all `for_each` iterations are processed by the same function object. Practically speaking, users cannot rely on accumulating any state modifications in a parallel `for_each` call.

```
struct callable
{
    void operator()(int& x)
    {
        ++x;
        ++i; // race here for parallel code
    }
    int get_i() const {
```

```
        return i;
    }
private:
    int i = 0;
};

callable c;

// serial for_each call
auto fun = std::for_each(v.begin(), v.end(), c);

// parallel for_each call
// The callable object cannot be read because parallel for_each version pu
std::for_each(std::execution::par, v.begin(), v.end(), c);

// for_each serial range version call
auto [_, fun] = std::ranges::for_each(v.begin(), v.end(), c);
```

We allow the same callable to be used in the proposed `std::ranges::for_each`.

```
// callable is used from the previous code snippet
callable c;
// The returned iterator is ignored
std::ranges::for_each(std::execution::par, v.begin(), v.end(), c);
```

Again, even though `c` accumulates state modifications, one cannot rely on that because an algorithm implementation is allowed to make as many copies of `c` as it wants. Of course, this can be overcome by using `std::reference_wrapper` but that might lead to data races.

```
// callable is used from the previous code snippet
// Wrapping a callable objection with std::reference_wrapper compiles, but
callable c;
std::ranges::for_each(std::execution::par, v.begin(), v.end(), std::ref(c))
```

Our conclusion is that it's user responsibility to provide such a callable that avoids data races, same as for C++17 parallel algorithms.

§ 2.9. `range` as an output

We would like to propose range as an output for the overloads that use range(s) for input. Similarly, we propose a sentinel for output where the input is passed as an iterator + sentinel. The reasons for that are:

- It creates a safer API where all the data sequences have known limits.
- More importantly, not for all algorithms the output size can be calculated based on the input size. An example is `copy_if` (and similar algorithms with *filtering* semantics), where the output sequence is allowed to be shorter than the input. Knowing the expected size of the output opens opportunities for more efficient parallel implementations.

See § 4 Proposed API for the examples.

There is already precedence in the standard that an algorithm takes two sequences and chooses the smaller size as the number of iterations it's going to make. The most telling example we were able to find is `std::ranges::transform`. For the record, `std::transform` (including the overload with execution policy) doesn't support different input sizes. Another example of an algorithm with potentially different input sizes is `std::mismatch` and `std::ranges::mismatch`.

The mentioned algorithms are not the exhaustive list. Sure, these support different sizes only for input sequences. However, for parallel algorithms having the output with its own size makes a lot of sense.

Alternatively, we can identify the family of `copy_if`-like algorithms and propose having `range` as an output only for them but from our perspective it would create even more inconsistency.

We can go even further and propose the overload with `range` for the algorithms like `for_each_n` or `generate_n` and have the similar semantics: whichever is smaller of the range size and the distance between `first` and `first + n`, it will be used to define the algorithm complexity.

§ 2.10. Parallel range algorithms are not customization points

We do not propose the parallel range algorithms to be customization points because it's unclear which parameter to customize for. One could argue that customizations may exist for execution policies, but we expect custom execution policies to become unnecessary once the C++ algorithms will work with schedulers/senders/receivers.

§ 2.11. `constexpr` parallel range algorithms

[P2902R0] suggests allowing algorithms with execution policies to be used in constant expressions. We do not consider that as a primary design goal for our work, however we will happily align with that proposal in the future once it progresses towards adoption into the working draft.

§ 3. More examples

§ 3.1. Less parallel algorithm calls and better expressiveness

Let's consider the following example:

```
reverse(policy, begin(data), end(data));
transform(policy, begin(data), end(data), begin(result), [](auto i){ return
auto res = any_of(policy, begin(result), end(result), pred);
```

It has three stages and eventually tries to answer the question if the input sequence contains an element after reversing and transforming it. The interesting considerations are:

- Since the example has three parallel stages, it adds extra overhead for parallel computation per algorithm.
- The first two stages will complete for all elements before the `any_of` stage is started, though it is not required for correctness. If reverse and transformation would be done on the fly, a good implementation of `any_of` might have skipped the remaining work when `pred` returns `true`, thus providing more performance.

Let's make it better:

```
// With fancy iterators
auto res = any_of(policy,
                  make_transform_iterator(make_reverse_iterator(end(data)),
                                          [](auto i){ return i * i; }),
                  make_transform_iterator(make_reverse_iterator(begin(data)
                                          [](auto i){ return i * i; }),
                  pred);
```

Now there is only one parallel algorithm call, and `any_of` can skip unneeded work. However, this variation also has interesting considerations:

- First, it doesn't compile. We use `transform iterator` to pass the transformation function, but the two `make_transform_iterator` expressions use two different lambdas, and the iterator type for `any_of` cannot be deduced because the types of `transform_iterator` do not match. One of the options to make it compile is to store a lambda in a variable.
- Second, it requires using a non-standard iterator.
- Third, the expressiveness of the code is not good: it is hard to read while easy to make a mistake like the one described in the first bullet.

Let's improve the example further with the proposed API:

```
// With ranges
auto res = any_of(policy, data | views::reverse | views::transform([](auto
                  pred);
```

The example above lacks the drawbacks described for the previous variations:

- There is only one algorithm call;
- The implementation might skip unnecessary work;
- There is no room for the lambda type mistake;
- The readability is much better compared to the second variation and not worse than in the first one.

§ 4. Proposed API

> NOTE: `std::ranges::for_each` is used as a reference point. When the design is ratified, it will be spread across other algorithms.

```
// for_each example
template <class ExecutionPolicy, random_access_iterator I, sentinel_for<I>
          class Proj = identity, indirectly_unary_invocable<projected<I, Pr
  ranges::for_each_result<I, Fun>
    ranges::for_each(ExecutionPolicy&& policy, I first, S last, Fun f, Proj
```

```
template <class ExecutionPolicy, random_access_range R, class Proj = identi
         indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
  ranges::for_each_result<ranges::borrowed_iterator_t<R>, Fun>
    ranges::for_each(ExecutionPolicy&& policy, R&& r, Fun f, Proj proj = {}

// binary transform example with range as an output and output sentinel
template< typename ExecutionPolicy,
          random_access_iterator I1, sized_sentinel_for<I1> S1,
          random_access_iterator I2, sized_sentinel_for<I2> S2,
          random_access_iterator O, sized_sentinel_for<O> O_Sentinel,
          copy_constructible F,
          class Proj1 = identity, class Proj2 = identity >
requires indirectly_writable<O,
             indirect_result_t<F&,
                               projected<I1, Proj1>,
                               projected<I2, Proj2>>>
constexpr binary_transform_result<I1, I2, O>
    transform( ExecutionPolicy&& policy, I1 first1, S1 last1, I2 first2, S2
               F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {} );

template< typename ExecutionPolicy,
          ranges::random_access_range R1,
          ranges::random_access_range R2,
          ranges::random_access_range RR,
          copy_constructible F,
          class Proj1 = identity, class Proj2 = identity >
requires indirectly_writable<ranges::iterator_t<RR>,
             indirect_result_t<F&,
                 projected<ranges::iterator_t<R1>, Proj1>,
                 projected<ranges::iterator_t<R2>, Proj2>>>
         && sized_sentinel_for<ranges::sentinel_t<R1>, ranges::iterator_t<R
         && sized_sentinel_for<ranges::sentinel_t<R2>, ranges::iterator_t<R
         && sized_sentinel_for<ranges::sentinel_t<R1>, ranges::iterator_t<R
constexpr binary_transform_result<ranges::borrowed_iterator_t<R1>,
                                  ranges::borrowed_iterator_t<R2>, ranges:
    transform( ExecutionPolicy&& policy, R1&& r1, R2&& r2, RR&& result, F b
               Proj1 proj1 = {}, Proj2 proj2 = {} );
```

§ 4.1. Possible implementation of a parallel range algorithm

```
// A possible implementation of std::ranges::for_each
namespace ranges
{
namespace __detail
{
struct __for_each_fn
{
    // ...
    // Existing serial overloads
    // ...

    // The overload for unsequenced and parallel policies. Requires random_
    template<class ExecutionPolicy, random_access_iterator I, sentinel_for<
             class Proj = identity, indirectly_unary_invocable<projected<I,
                 requires is_execution_policy_v<std::remove_cvref_t<Executi
    ranges::for_each_result<I, Fun>
    operator()(ExecutionPolicy&& exec, I first, S last, Fun f, Proj proj =
    {
        // properly handle execution policy; for the reference, a serial
        // implementation is provided
        for (; first != last; ++first)
        {
            std::invoke(f, std::invoke(proj, *first));
        }
        return {std::move(first), std::move(f)};
    }

    template<class ExecutionPolicy, random_access_range R, class Proj = ide
             indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun
    ranges::for_each_result<ranges::borrowed_iterator_t<R>, Fun>
    operator()(ExecutionPolicy&& exec, R&& r, Fun f, Proj proj = {}) const
    {
        return (*this)(std::forward<ExecutionPolicy>(exec), std::ranges::be
                       std::ranges::end(r), f, proj);
    }
}; // struct for_each
} // namespace __detail
inline namespace __for_each_fn_namespace
{
```

```
inline constexpr __detail::__for_each_fn for_each;
} // __for_each_fn_namespace
} // namespace ranges
```

§ 5. Absence of some serial range-based algorithms

We understand that some useful algorithms do not yet exist in `std::ranges`, for example, most of generalized numeric operations [numeric.ops]. The goal of this paper is however limited to adding overloads with `ExecutionPolicy` to the existing algorithms in `std::ranges` namespace. Any follow-up paper that adds `<numeric>` algorithms to `std::ranges` should also consider adding dedicated overloads with `ExecutionPolicy`.

§ 6. Further exploration

§ 6.1. Thread-safe views examination

We need to understand better whether using some `views` with parallel algorithms might result in data races. While some investigation was done by other authors in [P3159R0], it's mostly not about the data races but about ability to parallelize processing of data represented by various views.

We need to invest more time to understand the implications of sharing a state between `view` and `iterator` on the possibility of data races. One example is `transform_view`, where iterators keep pointers to the function object that is stored in the view itself.

Here are questions we want to answer (potentially not a complete list):

- Do users have enough control to guarantee absence of data races for such views?
- Are races not possible because of implementation strategy chosen by standard libraries?
- Do we need to add extra requirements towards thread safety to the standard views?

§ 7. Revision history

§ 8. R0 => R1

- Address the feedback from SG1 and SG9 review
- Add more information about iterator constraints
- Propose `range` as an output for the algorithms
- Require ranges to be bounded

§ 9. Polls

§ 9.1. SG9, Tokyo 2024

Poll 1: `for_each` shouldn't return the callable

SF	F	N	A	SA
2	4	2	0	0

Poll 2: Parallel `std::ranges` algos should return the same type as serial `std::ranges` algos

Unanimous consent.

Poll 3: Parallel ranges algos should require `forward_range`, not `random_access_range`

SF	F	N	A	SA
3	2	3	1	1

Poll 4: Range-based parallel algos should require const operator()

SF	F	N	A	SA
0	7	2	0	0

§ References

§ Informative References

[P2300R9]

Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. *`std::execution`*. 2 April 2024. URL: https://wg21.link/p2300r9

[P2500R2]

Ruslan Arutyunyan, Alexey Kukanov. *C++ parallel algorithms and P2300*. 15 October 2023. URL: https://wg21.link/p2500r2

[P2902R0]

Oliver Rosten. *constexpr 'Parallel' Algorithms*. 17 June 2023. URL: https://wg21.link/p2902r0

[P3136R0]

Tim Song. *Retiring niebloids*. 15 February 2024. URL: https://wg21.link/p3136r0

[P3159R0]

Bryce Adelstein Lelbach. *C++ Range Adaptors and Parallel Algorithms*. 18 March 2024. URL: https://wg21.link/p3159r0

[P3179R0]

Ruslan Arutyunyan, Alexey Kukanov. *C++ parallel range algorithms*. 15 March 2024. URL: https://wg21.link/p3179r0

[P3300R0]

Bryce Adelstein Lelbach. *C++ Asynchronous Parallel Algorithms*. 15 February 2024. URL: https://wg21.link/p3300r0